

Evaluation von Prompt-Engineering-Techniken für LLM-as-a-Judge: Second-Level Judging und Dynamic Prompting

Semesterprojekt-Proposal

Esmo Morina, Tatiana Malkina

5. Mai 2026

1 Introduction

1.1 Bereich der Arbeit

Large Language Models (LLMs) haben sich zu einer zentralen Technologie entwickelt. Dadurch steigt der Bedarf an skalierbaren Evaluationsverfahren, da menschliche Evaluationen zwar qualitativ hochwertig sind, jedoch mit erheblichem Zeit- und Kostenaufwand verbunden sind [1].

Ein bedeutender Forschungsbereich ist daher **LLM-as-a-Judge**, bei dem Sprachmodelle automatisch die Antworten anderer Modelle bewerten oder Systeme miteinander vergleichen. Solche Ansätze werden zunehmend für Benchmarking, Qualitätskontrolle und automatisierte Modellvergleiche genutzt. Die Qualität dieser Bewertungen hängt jedoch nicht nur vom eingesetzten Modell, sondern auch stark vom Prompt-Design ab [2], [3].

Ein aktueller Ansatz ist das **Second-Level Judging**, bei dem ein zweites Judge-Modell die Entscheidung und Begründung eines ersten Judges überprüft, um Fehler zu erkennen und die Zuverlässigkeit zu erhöhen. Erste Arbeiten berichten Verbesserungen durch diesen zweistufigen Prozess, insbesondere bei der Reduktion fehlerhafter Urteile [4]. Neuere Studien zeigen jedoch widersprüchliche Ergebnisse und kommen zu dem Schluss, dass Second-Level Judges die Gesamtleistung in vielen Fällen sogar verschlechtern können [3].

Parallel dazu zeigen Studien in anderen Bereichen, etwa bei der automatisierten Codefehlererkennung, dass **Dynamic Prompting** die Leistung von Sprachmodellen deutlich verbessern kann [5]. Ob sich dieser Ansatz auf LLM-as-a-Judge-Systeme übertragen lässt, wurde bislang kaum untersucht.

Zusätzlich zu allgemeinen Evaluationsrahmen und Benchmarks für LLM-basierte Bewertungsmodelle, etwa JudgeBench **judgebench2024**, konzentriert sich diese Arbeit speziell auf den Einfluss von Prompting-Strategien innerhalb solcher Bewertungs-Setups.

Ziel dieser Arbeit ist es, Prompting-Techniken für **LLM-as-a-Judge-Systeme** systematisch zu vergleichen. Im Fokus stehen dabei **Second-Level Judging** und **Dynamic Prompting**.

...

1.2 Motivation

Mit der zunehmenden Integration großer Sprachmodelle (LLMs) in produktive Systeme wird die zuverlässige Bewertung ihrer Ausgaben zu einer zentralen Herausforderung. Fehlbewertungen wirken sich dabei nicht nur auf Kosten und Ressourceneinsatz aus, sondern können direkt die Qualität realer Anwendungen beeinträchtigen – etwa durch falsche oder irreführende Antworten. Trotz der Vielzahl bestehender Ansätze bleibt die Entwicklung von Evaluationsverfahren, die gleichzeitig skalierbar, reproduzierbar und robust gegenüber variierenden Bedingungen sind, ein ungelöstes Kernproblem.

Vor diesem Hintergrund hat sich das Paradigma LLM-as-a-Judge als vielversprechender Ansatz etabliert, bei dem Sprachmodelle selbst zur automatisierten Bewertung von Modellantworten eingesetzt werden. Während dieser Ansatz eine effiziente Skalierung von Evaluationsprozessen ermöglicht, wirft er zugleich Fragen hinsichtlich der Zuverlässigkeit der erzeugten Bewertungen auf. Aktuelle Studien zeigen, dass die Ergebnisse stark von Faktoren wie Prompt-Formulierung, Modellwahl und Aufgabenstruktur abhängen, was eine hohe Variabilität verursacht und die praktische Einsetzbarkeit einschränkt [2], [3].

Ein naheliegender Ansatz zur Verbesserung der Bewertungsqualität ist **Second-Level Judging**, bei dem eine initiale Bewertung durch einen weiteren Prüfungsschritt verifiziert wird. Dahinter steht die Annahme, dass diese nachgelagerte Analyse die Zuverlässigkeit der Ergebnisse maßgeblich steigern kann.

Ähnliche Prinzipien finden sich in Ansätzen zur Selbstreflexion und iterativen Bewertung, die in anderen Kontexten eine Steigerung der Antwortqualität belegen konnten [6], [7]. Allerdings verdeutlichen empirische Ergebnisse im Bereich der automatisierten Evaluation, dass diese Effekte nicht konsistent auftreten [3], was die Notwendigkeit einer weiterführenden systematischen Analyse unterstreicht.

Parallel dazu nimmt das Prompt Engineering eine Schlüsselrolle ein, da es das Verhalten von Sprachmodellen maßgeblich determiniert. Insbesondere adaptive Ansätze wie das Dynamic Prompting konnten in diversen Anwendungsszenarien bereits signifikante Steigerungen der Leistungsfähigkeit erzielen. Inwieweit sich diese Effekte auf die Verlässlichkeit von LLM-basierten Evaluationsverfahren übertragen lassen, bleibt jedoch eine offene Forschungsfrage. Des Weiteren ist zu untersuchen, ob gezielte Prompting-Strategien es ermöglichen, auch mit kleineren und kosteneffizienteren Modellen eine äquivalente Bewertungsqualität zu erreichen.

Vor diesem Hintergrund verfolgt dieses Projekt das Ziel, die Qualität, Stabilität und Effizienz von LLM-basierten Bewertungssystemen systematisch zu untersuchen. Im Fokus steht dabei die Analyse von Second-Level Judging und Dynamic Prompting sowie deren Wechselwirkung mit unterschiedlichen Modellarchitekturen. Ziel ist es, die maßgeblichen Einflussfaktoren auf die Zuverlässigkeit automatisierter Evaluationen zu identifizieren und so eine fundierte Grundlage für deren praktischen Einsatz in produktiven Systemen zu schaffen.

...

2 Verwandte Forschung

Die Nutzung großer Sprachmodelle als Evaluatoren (LLM-as-a-Judge) wird zunehmend als skalierbare Alternative zu klassischen Bewertungsmethoden betrachtet. Studien belegen, dass leistungsstarke Modelle in der Lage sind, die Ausgaben anderer Sprachmodelle automatisiert zu validieren und dabei eine hohe Korrelation mit menschlichen Urteilen erreichen [2]. Darauf aufbauend wurden Verfahren entwickelt, die mittels strukturierter Prompt-Strategien eine präzisere Ausrichtung an menschlichen Bewertungskriterien anstreben [8]. In der Literatur wird dieser Ansatz häufig als parametrisierte Bewertungsfunktion formalisiert [1]:

$$\text{Score} = f(\text{model}, \text{prompt}, \text{task}, \text{context}) \quad (1)$$

Die Bewertungsqualität hängt dabei maßgeblich vom gewählten Modell, dem Prompt, der spezifischen Aufgabe sowie dem Kontext ab und reagiert entsprechend sensitiv auf Variationen dieser Faktoren. Parallel dazu werden Methoden untersucht, die die Qualität von Modellantworten durch stärker strukturierte Generationsprozesse steigern. Da die Güte der generierten Antworten die Grundlage für deren Evaluation bildet, können solche Ansätze indirekt auch die Leistungsfähigkeit von LLM-basierten Evaluatoren beeinflussen. Ein prominentes Beispiel ist das Dynamic Prompting, bei dem zunächst eine Zwischenanalyse erzeugt und anschließend als kontextuelle Erweiterung in den Prompt integriert wird. Während empirische Ergebnisse zeigen, dass diese Methode die Antwortqualität signifikant verbessert, wurde ihr Einfluss auf Aufgaben der automatisierten Evaluation (LLM-as-a-Judge) bislang nur unzureichend untersucht [5].

Daneben existieren mehrstufige Bewertungsverfahren wie Second-Level Judging, bei dem eine Primärbewertung erneut analysiert und gegebenenfalls korrigiert wird. Ziel ist die Steigerung der Reliabilität des Urteils. Empirische Studien belegen jedoch, dass solche Verfahren keine konsistente Verbesserung garantieren und in spezifischen Modell-Prompt-Konfigurationen sogar kontraproduktive Effekte erzielen können [3].

Neuere Arbeiten erweitern diese Idee zum Meta-Judging, bei dem Modelle nicht nur Antworten, sondern auch bereits erzeugte Evaluationen prüfen und somit als „Richter der Richter“ fungieren. Diese Ansätze zielen darauf ab, die Konsistenz automatisierter Bewertungssysteme zu erhöhen, und zeigen in isolierten Konfigurationen bereits Leistungsgewinne. Bestehende Studien konzentrieren sich jedoch häufig auf komplexe Architekturen wie Multi-Agenten-Systeme oder präferenzbasiertes Training, während praktisch leichter umsetzbare Varianten wie das Second-Level Judging bislang unzureichend erforscht sind [9].

Insgesamt zeigt sich, dass zusätzliche Analyseebenen – sowohl innerhalb des Reasoning-Prozesses als auch durch separate Prüfungsschritte – die Entscheidungsqualität maßgeblich beeinflussen können, deren exakte Wirkweise jedoch noch nicht eindeutig geklärt ist. Insbesondere bleibt offen, unter welchen Bedingungen Second-Level-Ansätze die Verlässlichkeit von LLM-basierten Evaluatoren tatsächlich optimieren. Dies macht eine systematische Untersuchung verschiedener Implementierungen – insbesondere dynamischer Prompting-Ansätze und mehrstufiger Verfahren – zu einer dringlichen Forschungsaufgabe.

...

3 Problemstellung

3.1 Beitrag der Arbeit

Im Rahmen dieser Arbeit wird die Zuverlässigkeit automatisierter Bewertungsverfahren auf Basis des Ansatzes *LLM-as-a-Judge* untersucht, wobei der Fokus insbesondere auf mehrstufigen Strategien liegt. Bestehende Studien liefern hierzu widersprüchliche Ergebnisse: Während mehrstufige Verfahren wie *Dynamic Prompting* die Qualität bei Analyseaufgaben (z. B. Codeanalyse und Schwachstellenerkennung) nachweislich steigern können, führen analoge Ansätze im Kontext der Evaluation nicht zwangsläufig zu den erwarteten Verbesserungen.

Insbesondere zeigt der Ansatz des *Second-Level Judging*, der eine zusätzliche Überprüfung initialer Bewertungen vorsieht, trotz seiner konzeptionellen Einfachheit häufig instabile oder sogar schlechtere Ergebnisse. Im Gegensatz dazu erzielen komplexere Methoden wie das *Meta-Judging* oftmals eine höhere Übereinstimmung in der Bewertung. Dieses Spannungsfeld stellt ein in der Literatur bislang unzureichend geklärtes Problem dar.

Ziel dieser Arbeit ist die systematische Untersuchung dieser Diskrepanzen sowie die Identifikation von Bedingungen, unter denen mehrstufige Ansätze die Zuverlässigkeit der Bewertung maßgeblich beeinflussen.

Zur Umsetzung wird eine experimentelle Pipeline für binäre Klassifikationsaufgaben entwickelt, die den systematischen Vergleich verschiedener Konfigurationen des Bewertungsprozesses ermöglicht. Dabei werden insbesondere der Einfluss von *Second-Level Judging* sowie verschiedener Prompting-Strategien – einschließlich *Dynamic Prompting* – auf die Stabilität und Konsistenz der Bewertungen analysiert. Zusätzlich erfolgt ein Vergleich von Modellen variierender Leistungsfähigkeit, um die Korrelation zwischen Bewertungsqualität und Rechenaufwand zu untersuchen.

Als Datengrundlage dienen etablierte Benchmarks, ergänzt durch einen dedizierten Satz eigener Beispiele. Zur Evaluation werden klassische Metriken (Precision, Recall, F1-Score), das Maß der Übereinstimmung (Cohen's Kappa) sowie Stabilitätskennzahlen auf Basis der Variabilität über mehrere Iterationen eingesetzt.

Die Ergebnisse der Arbeit sollen dazu beitragen, die Kausalmechanismen hinter den beobachteten Diskrepanzen zu isolieren und evidenzbasierte Empfehlungen für den Einsatz mehrstufiger Bewertungsansätze abzuleiten.

...

3.2 Forschungsfragen

Zur Beantwortung dieses Ziels ergibt sich die folgende **Hauptfrage**:

Wie beeinflussen mehrstufige Bewertungsansätze wie *Second-Level Judging* und *Dynamic Prompting* die Qualität, Stabilität und Effizienz von LLM-basierten Bewertungssystemen im Vergleich zu einem einstufigen Baseline-Ansatz?

Teilfragen

1. Wie verhält sich *Second-Level Judging* im Vergleich zu *Dynamic Prompting* sowie zu einem einstufigen Baseline-Ansatz (*Single-Prompt Baseline*) unter vergleichbaren Bedingungen hinsichtlich der Stabilität und Konsistenz der Bewertungsergebnisse?
2. In welchem Ausmaß führen mehrstufige Ansätze (insbesondere *Second-Level Judging* und *Dynamic Prompting*) zu einer Verbesserung der Bewertungsqualität im Vergleich zum Baseline-Ansatz?
3. Welchen Einfluss haben unterschiedliche Datensätze und Modelltypen auf die Bewertungsqualität von *Second-Level Judging*, *Dynamic Prompting* und dem Baseline-Ansatz?
4. Inwiefern beeinflussen Variationen in der Prompt-Formulierung die Bewertungsergebnisse von *Second-Level Judging*?
5. Unter welchen Konfigurationen von Modellen, Prompts und mehrstufigen Bewertungsprozessen ergeben sich Verbesserungen oder Verschlechterungen sowohl der Bewertungsergebnisse als auch der Effizienz (insbesondere Kosten und Laufzeit)?

4 Vorgehensweise

Im Rahmen dieser Arbeit wird zunächst eine Literaturrecherche zu bestehenden Ansätzen im Bereich *LLM-as-a-Judge* durchgeführt. Dabei werden insbesondere Single-Prompt Baselines, Second-Level Judging und Dynamic Prompting betrachtet. Ziel ist es, ein grundlegendes Verständnis dieser Methoden zu gewinnen und geeignete Benchmarks auszuwählen.

Anschließend werden passende Benchmarks definiert, wobei der Fokus auf sicherheitsrelevanten binären Klassifikationsaufgaben liegt. Dazu gehören beispielsweise die Unterscheidung zwischen sicheren und unsicheren Antworten sowie zwischen korrekten und inkorrekten Inhalten. Zusätzlich können auch manipulierte Eingaben, wie etwa Prompt Injection, berücksichtigt werden. Neben bestehenden Datensätzen wird auch ein eigener Benchmark erstellt.

Darauf aufbauend wird ein experimentelles Vergleichsdesign umgesetzt, in dem die verschiedenen Ansätze miteinander verglichen werden. Alle Methoden werden unter gleichen Bedingungen auf denselben Daten getestet. Außerdem werden verschiedene Prompt-Varianten sowie mehrere Durchläufe genutzt, um die Stabilität der Ergebnisse zu untersuchen.

Für die Experimente werden mehrere Sprachmodelle verwendet, darunter frei verfügbare Modelle sowie eine leistungsstärkere Referenz. So kann analysiert werden, welchen Einfluss die Wahl des Modells auf die Ergebnisse hat.

Die Bewertung erfolgt anhand verschiedener Metriken. Dazu zählen Accuracy und F1-Score zur Messung der Qualität sowie einfache Kennzahlen zur Stabilität, wie die Abweichung zwischen mehreren Durchläufen. Zusätzlich werden Effizienz (z. B. Laufzeit und Anzahl der Aufrufe) sowie der Tokenverbrauch und die daraus entstehenden Kosten betrachtet. Um die Reproduzierbarkeit sicherzustellen, wird der gesamte Ablauf automatisiert. Dabei werden alle wichtigen Informationen

wie Prompts, Antworten und Einstellungen gespeichert, sodass die Experimente nachvollzogen und wiederholt werden können.

...

5 Planung

5.1 Eigene Vorkenntnisse

Im Rahmen des Studiums wurden grundlegende Kenntnisse in der Programmierung mit Python erworben, die insbesondere für die Umsetzung der experimentellen Auswertung genutzt werden. Darüber hinaus bestehen erste Erfahrungen im Bereich Machine Learning, die im Rahmen von Lehrveranstaltungen und Praktika, insbesondere im Bereich Deep Learning, gesammelt wurden.

Diese Vorkenntnisse umfassen grundlegende Konzepte wie das Training und die Evaluation von Modellen sowie den Umgang mit Datensätzen. Zudem wurden erste Erfahrungen im Umgang mit Schnittstellen (APIs) im Rahmen von Programmieraufgaben, unter anderem in Java, gesammelt.

Für die vorliegende Arbeit sind insbesondere die Fähigkeiten zur Datenverarbeitung, zur Implementierung von Experimenten sowie zur strukturierten Auswertung von Ergebnissen relevant. Diese werden im Rahmen der Arbeit weiter angewendet und vertieft.

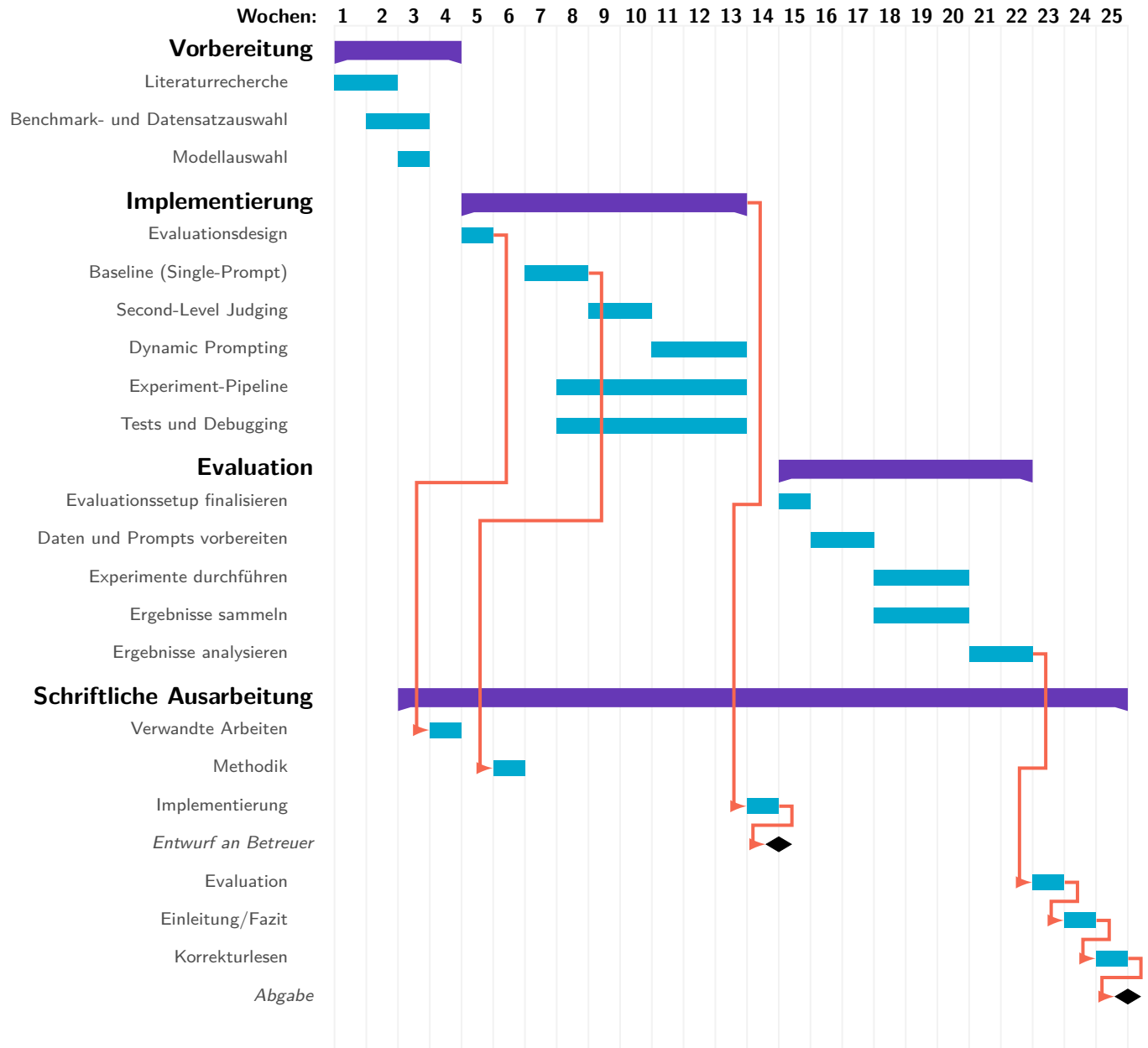
Zusätzlich werden im Verlauf der Arbeit neue Kenntnisse erworben, insbesondere im Bereich der Bewertung von Large Language Models, fortgeschrittener Prompting-Strategien (z. B. Second-Level Judging und Dynamic Prompting) sowie der systematischen Durchführung und Analyse von Benchmark-Experimenten. Auch Aspekte wie die Automatisierung von Experimenten und die Analyse von Kosten und Effizienz werden dabei eine wichtige Rolle spielen.

...

5.2 Benötigte Ressourcen

- **Laptop:** Durchführung der Experimente sowie Datenverarbeitung und Auswertung.
- **Python und Bibliotheken:** Implementierung der Experimente, Verarbeitung der Daten und Analyse der Ergebnisse.
- **Zugriff auf Large Language Models (APIs):** Nutzung leistungsstarker Sprachmodelle für die Durchführung der Evaluation; teilweise kostenpflichtig.
- **Öffentliche Datensätze:** Grundlage für die Durchführung der Benchmark-Experimente.
- **Eigener Benchmark:** Ergänzende Datensätze zur Untersuchung spezifischer Fragestellungen.
- **Budget für API-Nutzung:** Abdeckung der Kosten für kostenpflichtige Modellzugriffe.

5.3 Zeitplanung



5.4 Risiken und Ausweichplan

- **Nicht verfügbare oder eingeschränkt nutzbare Ressourcen (LLM-APIs)**
Risiko: Budgetüberschreitungen, Rate Limits oder eingeschränkter Zugriff auf Modelle.
Ausweichplan: Nutzung kostengünstiger oder frei verfügbarer Modelle, Reduktion der Stichprobengröße sowie Caching von Ergebnissen.
- **Instabilität und geringe Reproduzierbarkeit von LLM-Ergebnissen**
Risiko: Hohe Varianz durch nicht-deterministisches Verhalten der Modelle.
Ausweichplan: Durchführung mehrerer Durchläufe, Fixierung von Parametern (z. B. Temperature) sowie Einsatz geeigneter Stabilitätsmetriken.
- **Keine signifikanten Verbesserungen durch mehrstufige Ansätze**
Risiko: Second-Level Judging und Dynamic Prompting liefern keine besseren Ergebnisse als die Baseline.
Ausweichplan: Analyse der Einflussfaktoren, Erweiterung der Evaluation um zusätzliche Metriken (z. B. Effizienz und Kosten) sowie qualitative Fallanalysen.
- **Verschlechterung der Ergebnisse durch Second-Level Judging**
Risiko: Zusätzliche Bewertungsschritte können zu inkonsistenten oder schlechteren Resultaten führen.
Ausweichplan: Anpassung der Prompt-Strategien, Vergleich mit einfacheren Methoden sowie Analyse der Ursachen für Verschlechterungen.
- **Hoher Implementierungsaufwand der Experiment-Pipeline**
Risiko: Verzögerungen durch technische Komplexität.
Ausweichplan: Iterative Entwicklung, modularer Aufbau sowie Nutzung bestehender Frameworks.
- **Ungeeignete oder nicht ausreichende Datensätze**
Risiko: Benchmarks decken die Forschungsfragen nicht ausreichend ab.
Ausweichplan: Kombination mehrerer Datensätze sowie Ergänzung durch einen eigenen Benchmark.
- **Kein klar überlegener Prompting-Ansatz identifizierbar**
Risiko: Es lässt sich kein eindeutig bester Ansatz bestimmen.
Ausweichplan: Analyse von Trade-offs (Qualität, Kosten, Stabilität) sowie Ableitung praxisnaher Empfehlungen.

Literatur

- [1] J. Gu, X. Jiang, Z. Shi u. a., "A Survey on LLM-as-a-Judge," *The Innovation*, S. 101253, Jan. 2026. DOI: 10.1016/j.xinn.2025.101253
- [2] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu u. a., "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," *arXiv preprint arXiv:2306.05685*, 2023.
- [3] T. Biskupski und S. Kleber, "Evaluating the Reliability and Fidelity of Automated Judgment Systems of Large Language Models," *arXiv preprint arXiv:2603.22214*, 2026.
- [4] M. Russinovich, A. Salem und R. Eldan, "Great, Now Write an Article About That: The Crescendo Multi-Turn LLM Jailbreak Attack," *arXiv preprint arXiv:2404.01833*, 2025.
- [5] L. Huynh u. a., "Detecting Code Vulnerabilities using LLMs," 2025.
- [6] A. Madaan u. a., "Self-Refine: Iterative Refinement with Self-Feedback," 2023. arXiv: 2303.17651 [cs.CL].
- [7] N. Shinn, F. Cassano, B. Labash, D. Gopinath, K. Narasimhan und S. Yao, "Reflexion: Language Agents with Verbal Reinforcement Learning," *arXiv preprint arXiv:2303.11366*, 2023.
- [8] Y. Liu, D. Iter, Y. Xu u. a., "G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment," *arXiv preprint arXiv:2303.16634*, 2023.
- [9] H. Silva, M. Mendes und H. G. Oliveira, "Meta-Judging with Large Language Models: Concepts, Methods, and Challenges," *arXiv preprint arXiv:2601.17312*, 2026.